Tema 5.2: Modelo Vista Controlador (MVC)

Pablo Gordillo (pabgordi@ucm.es)

Departamento de Sistemas Informáticos y Computación

Facultad de Informática



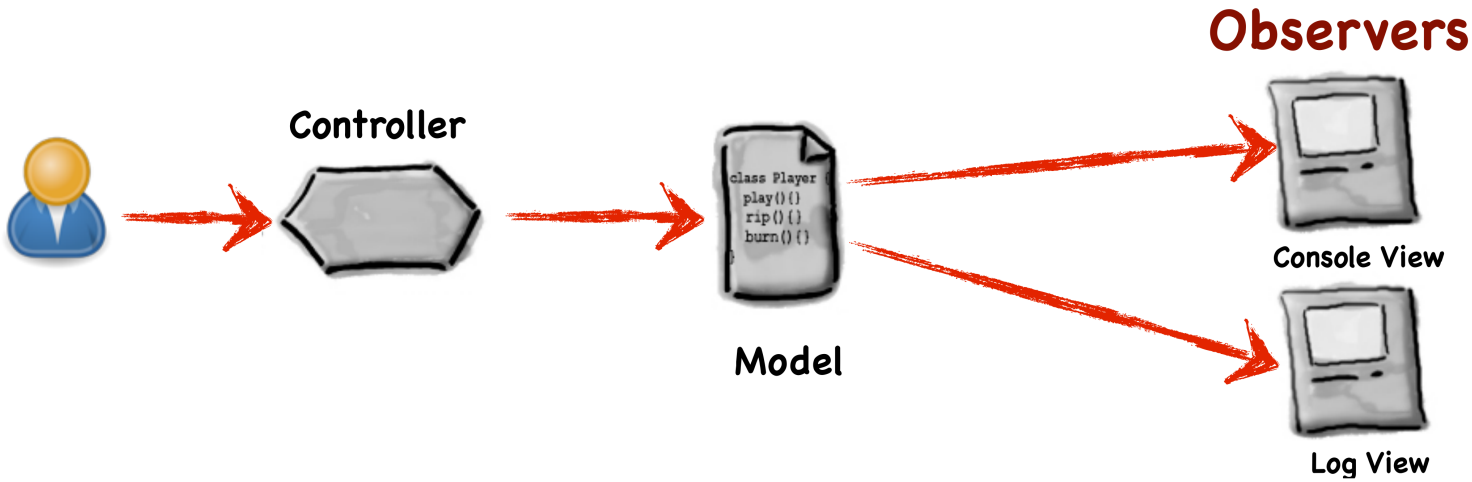
(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

El Modelo Vista Controlador (MVC)

- El modelo vista controlador es un **patrón de diseño de software**, es decir, un esquema común de programación cuyo objetivo es poder reutilizar el diseño de otros sistemas para programar nuevas aplicaciones. Sus orígenes datan de 1979, para Smaltalk.
- El MVC se utiliza en el desarrollo de interfaces gráficas de usuario y aplicaciones web. Existen distintas instancias del modelo.
- Las vistas no tienen por que ser una interfaz gráfica. Puede ser una consola, o generar información en un fichero de log.

El Modelo Vista Controlador (MVC)

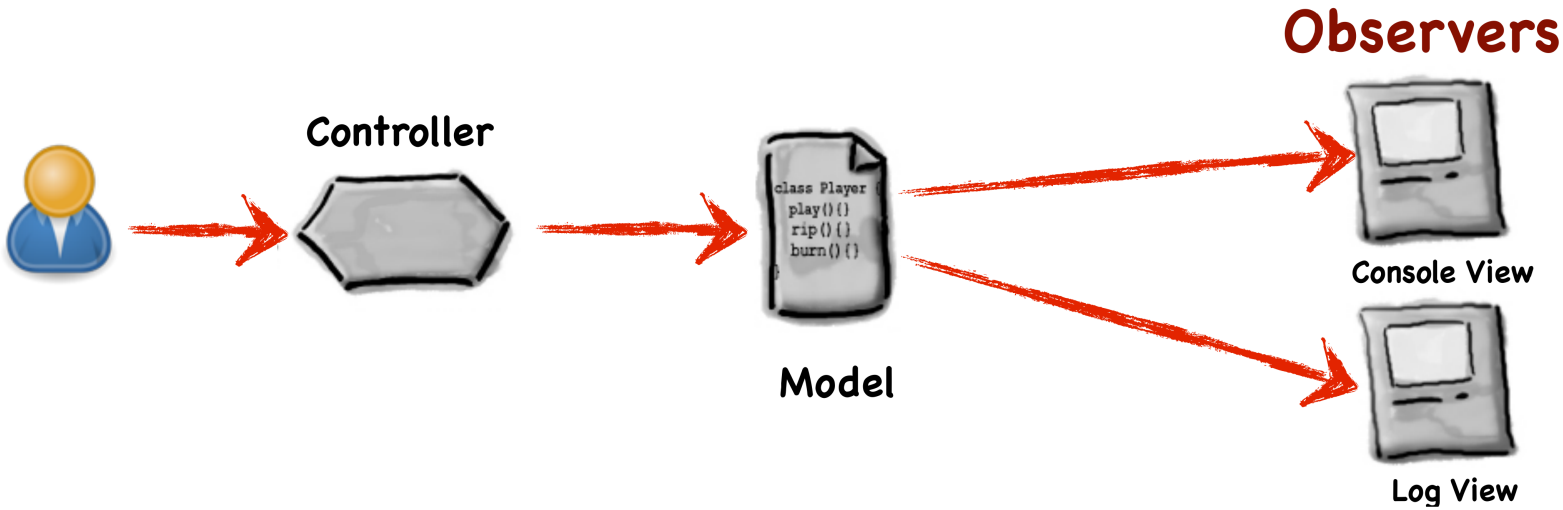
- En la instancia más general del modelo (sin interfaz gráfica) el usuario interacciona con el controlador.



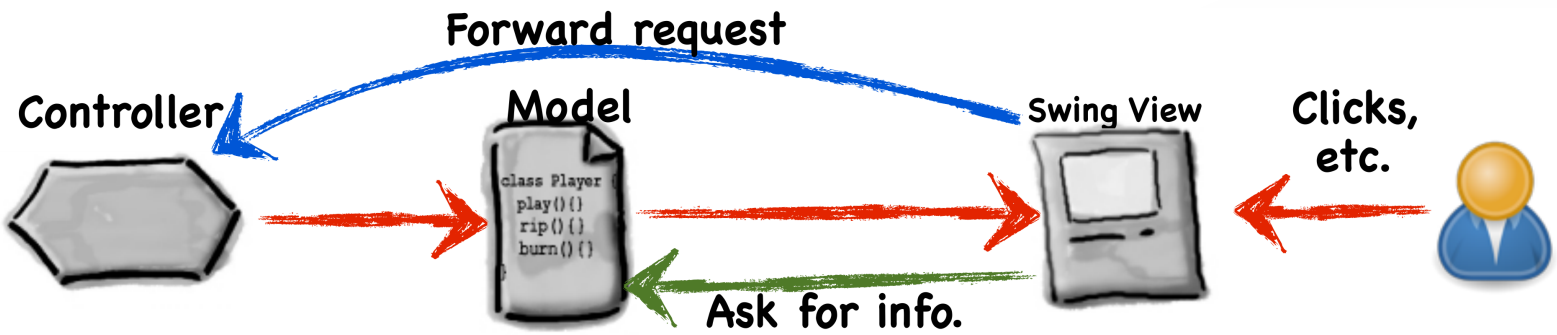
- El usuario dice al **Controlador** que quiere hacer...
- El **Controlador** sabe cómo pasar instrucciones del usuario al **modelo**, por ejemplo llamando a sus métodos...
- El **modelo notifica** las **vistas** cuando cambia su estado...
- Las **vistas** reaccionan al recibir **notificaciones**, por ejemplo escribiendo el estado en la consola o en un fichero...
- Se implementa usando el patrón Observable

El Modelo Vista Controlador (MVC)

- Cuando construimos una vista utilizando Swing, puede ser más difícil entender la separación entre el controlador y la vista.
- Hemos dicho que el usuario interactúa con el controlador, pero si desarrollamos una vista usando Swing un click sobre un botón se captura en un Action Listener que forma parte de la clase de la vista.
- ¿Cómo resolvemos esta confusión?

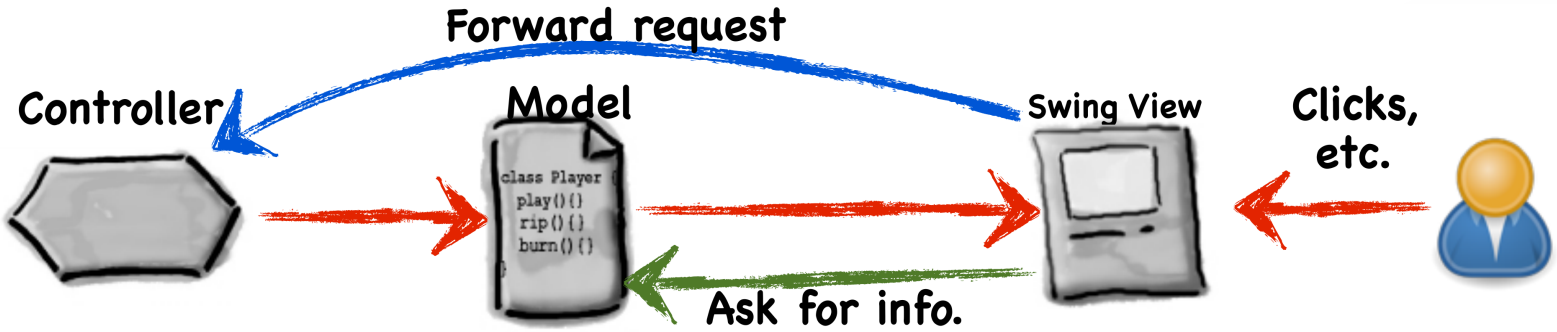


El Modelo Vista Controlador (MVC)



- **El modelo**: parte lógica del programa, la que resuelve el problema, y la que genera el resultado. Por tanto contiene los datos y funcionalidades de la aplicación.
- **El controlador**: Comunica el modelo y la vista. Recibe los datos introducidos por el usuario a través de la vista y se lo envía al modelo para que realice las operaciones correspondientes.
- **La vista**: Es la parte que interactua con el usuario (parte gráfica) y que envía los datos al controlador para que se realice la comunicación con el modelo.

El Modelo Vista Controlador (MVC)



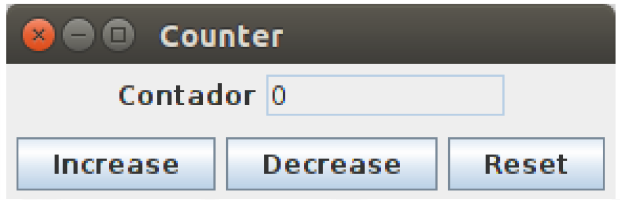
- Los Action Listeners reenvían las peticiones del usuario al controlador que a su vez las realizará sobre el modelo.
- A veces permitimos que los Action Listeners verifiquen los datos proporcionados por el usuario antes de enviarlos al controlador, de esta manera en controlador no tiene que preocuparse de mostrar errores mediante dialogs, etc.
- ¿Entonces para qué necesitamos el controlador? Es una abstracción útil de las operaciones que se pueden realizar sobre el modelo y de la comunicación con el modelo.
- A veces, permitimos que la vista accede al modelo, pero solo para pedir información y no para realizar operaciones.

El Modelo Vista Controlador (MVC)

- La separación entre el modelo, la vista y el controlador permite que cada una de las componentes pueda evolucionar por separado, sin necesidad de modificar las otras.
  - **Vista:** Contiene como atributo al controlador. Puede haber varias vistas.
    - Informa al controlador de los datos introducidos por el usuario.
    - Le pide al controlador que la registre como **observador**
  - **Controlador:** Contiene como atributo al modelo. Puede haber varios controladores.
    - Pasa los datos de la vista al modelo.
    - Contiene métodos para **añadir y quitar observadores** (vistas).
  - **Modelo:** Contiene como atributo una **lista de observadores**. El modelo es único.
    - Implementa la lógica de la aplicación.
    - Avisa a todos los observadores cuando se producen cambios en el modelo.

Un ejemplo para entender todas las partes

- Vamos a diseñar una aplicación que permite actualizar el valor de un contador. Concretamente podremos incrementar. decrementar e inicializar el contador. La GUI es la siguiente:



- La vista (GUI) contiene tres botones. Cuando se pulsa un botón, la información sobre la acción se manda al controlador. El controlador ordena al modelo que realice la operación correspondiente.
- EL modelo simplemente realiza la operación y avisa a las vistas del cambio efectuado a través de los observadores (ten en cuenta que la vista es un observador y por tanto se avisa a la vista del cambio).

Los observadores

- Como las vistas tienen que registrarse como observadores del modelo, una vista debe contener operaciones a través de las cuales se comunica con el modelo.
- Como el modelo actualiza un contador, entonces debe avisar a las vistas (observadores) de dicha actualización. Por lo tanto vamos a usar la siguiente interfaz para que la implementen los observadores:

```
public interface CounterObserver{
  //para que el modelo pueda avisar ala vista de un cambio
  //en el contador

  public void update(int d);
}
```

Los observadores

- Por otro lado el controlador debe suministrar métodos que permitan añadir y eliminar observadores al modelo. Por lo tanto hay que garantizar que el modelo contiene métodos para añadir y eliminar observadores a su lista de observadores. Para ello **obligaremos al modelo a implementar la siguiente interfaz:**

```
public interface ModelObservable<T>{

  public void addObserver(T o);
  public void removeObserver(T o);
}
```

La interfaz es genérica para poder aplicarse a cualquier tipo de observador. En este caso la instanciaremos con CounterObserver

El modelo

```
public class CounterModel {
  private int counter;


  public CounterModel() {
    this.counter = 0;

  }

  public void increaseCounter() {
    this.counter++;

  }

  public void decreaseCounter() {
    this.counter--;

  }

  public void reset() {
    this.counter = 0;

  }
  ...
```

El modelo

```
public class CounterModel {
  private int counter;
  private List <CounterObserver> observers;


  public CounterModel() {
    this.counter = 0;
    this.observers = new ArrayList<CounterObserver>();
  }

  public void increaseCounter() {
    this.counter++;

  }

  public void decreaseCounter() {
    this.counter--;

  }

  public void reset () {
    this.counter = 0;

  }
  ...
```

El modelo

```
public class CounterModel {
  private int counter;
  private List <CounterObserver> observers;


  public CounterModel() {
    this.counter = 0;
    this.observers = new ArrayList<CounterObserver>();
  }

  public void increaseCounter() {
    this.counter++;
    this.notifyObservers (this.counter);
  }

  public void decreaseCounter() {
    this.counter--;
    this.notifyObservers (this.counter);
  }

  public void reset () {
    this.counter = 0;
    this.notifyObservers (this.counter);
  }
  ...
```

El modelo envía notificaciones a los observadores (vistas)

El modelo

```
public class CounterModel {
  private void notifyObservers (int n) {
    for (CounterObserver ob : observers )
    ob.update(n);
  }

}
```

El modelo

```
public class CounterModel implements ModelObservable<CounterObserver> {
  private void notifyObservers (int n) {
    for (CounterObserver ob : observers )
    ob.update(n);
  }

  @Override
  public void addObservador(CounterObserver o) {
    this . observers . add(o);
  }

  @Override
  public void removeObservador(CounterObserver o) {
    this . observers . remove(o);
  }
}
```

El modelo permite añadir y eliminar observadores de su lista

La vista

```
public class View1 extends JFrame {

  private JTextField textField ;

  public View1(                ) {
    super("Counter");

    this .initGUI();


  }

  private void initGUI() {
    ...
    JButton increase = new JButton("Increase");



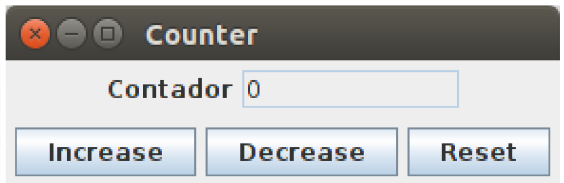






  }


}
```



La vista

```
public class View1 extends JFrame {
  private Controller control;
  private JTextField textField;

  public View1(Controller c) {
    super("Counter");
    this.control = c;
    this.initGUI();
    this.control.addObserver(this);
    // la vista se añade como observadora
  }

  private void initGUI() {
    ...
    JButton increase = new JButton("Increase");



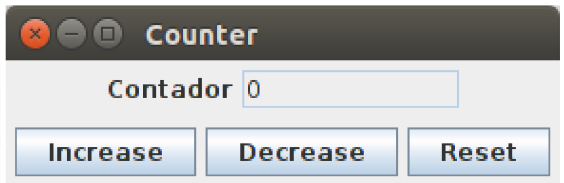






  }


}
```



La vista

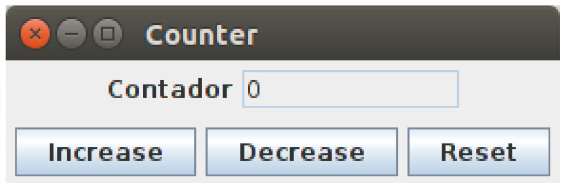
```
public class View1 extends JFrame implements CounterObserver {
  private Controller control;
  private JTextField textField;

  public View1(Controller c) {
    super("Counter");
    this.control = c;
    this.initGUI();
    this.control.addObserver(this);

  }

  private void initGUI() {
    ...
    JButton increase = new JButton("Increase");
    increase.addActionListener(new ActionListener() {
      @Override
      public void actionPerformed(ActionEvent arg0) {
        control.increase();
      }
    });
  }


}
```



Cada cambio que sufre una componente supone avisar al controlador

La vista

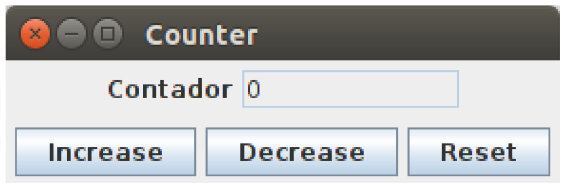
```
public class View1 extends JFrame implements CounterObserver {
  private Controller control;
  private JTextField textField;

  public View1(Controller c) {
    super("Counter");
    this.control = c;
    this.initGUI();
    this.control.addObserver(this);

  }

  private void initGUI() {
    ...
    JButton increase = new JButton("Increase");
    increase.addActionListener(new ActionListener() {
      @Override
      public void actionPerformed(ActionEvent arg0) {
        control.increase();
      }
    });
  }

  @Override //Acción a realizar cuando le notifican un cambio
  public void update(int d) { textField.setText(d+""); }
}
```



El Controlador

```
public class Controller {
  private CounterModel model;

  public Controller (CounterModel m) {
    this.model = m;
  }









}
```

El controlador necesita el modelo para poder ordenarle que realice las operaciones pertinentes

El Controlador

```
public class Controller {
  private CounterModel model;

  public Controller (CounterModel m) {
    this.model = m;
  }

  public void increase () {
    this.model.increaseCounter();
  }

  public void reset () {
    this.model.reset();
  }

  public void decrease() {
    this.model.decreaseCounter();
  }

}
```

Por cada evento que pueda generar la vista, el controlador tendrá un método asociado

El Controlador

```
public class Controller {
  private CounterModel model;

  public Controller (CounterModel m) {
    this.model = m;
  }

  public void increase () {
    this.model.increaseCounter();
  }

  public void reset () {
    this.model.reset();
  }

  public void decrease() {
    this.model.decreaseCounter();
  }

  public void addObserver(CounterObserver ob) {
    this.model.addObserver(ob);
  }

  public void removeObserver(CounterObserver ob) {
    this.model.removeObserver(ob);
  }
}
```

Contiene métodos para añadir y eliminar observadores

El Método main

```
public class Main1 {
  public static void main(String[] args) {




  }
}
```

El Método main

```
public class Main1 {
  public static void main(String[] args) {
    CounterModel model = new CounterModel();
    Controller controller = new Controller(model);


  }
}
```

Se crean el modelo y el controlador

El Método main

```
public class Main1 {
  public static void main(String[] args) {
    CounterModel model = new CounterModel();
    Controller controller = new Controller(model);
    EventQueue.invokeLater(new Runnable() {
      public void run() {
        new View1(controller);
      }
    });
  }
}
```

Se crea la vista pasándole el controlador

Resumen: MVC

- Patrón de diseño que contiene tres componentes principales:
  - **Modelo**: Contiene los datos y funcionalidades del dominio de la aplicación.
  - **Vista**: Representación gráfica de la aplicación. Puede ser gráfica o basada en texto.
  - **Controlador**: Interpreta las acciones del usuario y las traduce en operaciones sobre el modelo.
- Esta estructura permite que cada componente pueda evolucionar por separado, de forma independiente al resto.
- En una aplicación existe un solo modelo, pero puede haber varios controladores y varias vistas.
- Hemos visto una instancia del modelo que incluye el patrón **Observer**.

Resumen: MVC

- **El modelo**
  - El modelo contiene **los datos y funcionalidades (operaciones) que se pueden realizar en la aplicación**.
  - Puede recibir **consultas** sobre su estado.
  - Puede recibir **solicitudes para cambiar su estado** (realizar operaciones sobre su estado).
  - El modelo **debe ser independiente de la vista y del controlador**: **no debe ver** las clases de los otros componentes.
    - Esto garantiza que se puede cambiar la vista y el controlador sin afectar al modelo.
    - Aun así, debe **notificar** a las vistas las modificaciones producidas en el modelo. Para ello se utiliza el patrón **Observer**.
  - El **controlador** debe tener acceso a las operaciones sobre el modelo.

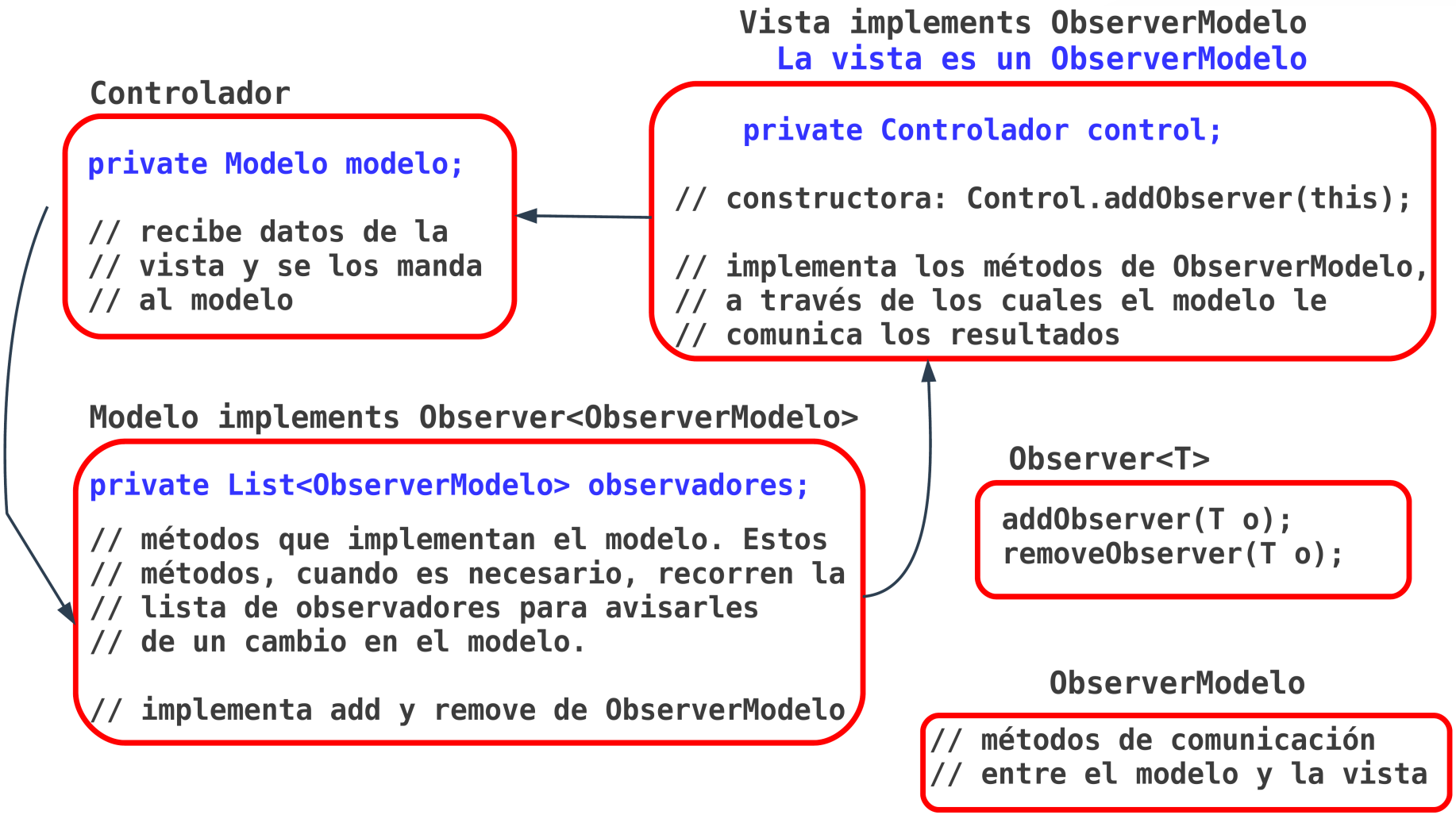
Resumen: MVC

- **La vista**
  - Son **representaciones visuales del modelo**.
  - Pueden estar formadas por componentes gráficos, pero no necesariamente.
  - Las vistas pueden ser una **representación parcial del modelo**: destacar algunos aspectos y ocultar otros.
  - Las vistas **no deben ver las clases del modelo**: es el modelo el que *notifica* a las vistas los cambios que se producen en su estado.
    - En algunas variantes del MVC se permite que la vista tenga **acceso de consulta** del modelo, sin posibilidad de hacer cambios.
  - Las vistas **sí deben tener acceso al controlador**. Así pueden indicar acciones del usuario que modifiquen el modelo.
  - Es posible tener **varias vistas simultáneamente** para un modelo.
  - Se pueden crear sin necesidad de modificar el modelo.

Resumen: MVC

- **El controlador**
  - El controlador **determina las modificaciones del modelo de acuerdo a interacciones con la vista**.
  - El controlador **recibe peticiones de la vista** y responde a ellas modificando el modelo.
  - El controlador **ve el modelo y modifica su estado invocando a los métodos necesarios**.
  - La relación del controlador con las vistas puede ser diferente según la variante del MVC que se utilice:
    - En algunas variantes el controlador contiene los **listeners** de las vistas.
    - Nosotros consideramos los listeners **como parte de las vistas**, que debe llamar a métodos del controlador.
    - De esta forma, un cambio en la vista no afecta al controlador.

Resumen: MVC


Vista implements ObserverModelo
La vista es un ObserverModelo
private Controlador control;
// constructora: Control.addObserver(this);
// implementa los métodos de ObserverModelo,
// a través de los cuales el modelo le
// comunica los resultados
Controlador
private Modelo modelo;
// recibe datos de la
// vista y se los manda
// al modelo
Modelo implements Observer<ObserverModelo>
private List<ObserverModelo> observadores;
// métodos que implementan el modelo. Estos
// métodos, cuando es necesario, recorren la
// lista de observadores para avisarles
// de un cambio en el modelo.
// implementa add y remove de ObserverModelo
Observer<T>
addObserver(T o);
removeObserver(T o);
ObserverModelo
// métodos de comunicación
// entre el modelo y la vista